CAHIER DES CHARGES :

Application web de gestion intégrée (ERP / CRM léger) : LogiStock SA

Projet intégré en Informatique de Gestion, Groupe 6

Version 1.0, octobre 2026

Équipe : Checkne KAMISSOKO, Diakaridia SIDIBE, Mohamed DOUCOURE, Harouna MAIGA, Moussa KONE, Aminata TANGARA, Djenebou TRAORE

1. Présentation du projet

1.1 Contexte

LogiStock SA est une entreprise commerciale de taille intermédiaire, spécialisée dans la distribution de fournitures pour professionnels (matériel de bureau, consommables, petit équipement). Son activité est aujourd'hui gérée avec des outils disparates : tableurs, cahiers papier et échanges d'e-mails. Ces outils sont fragiles et peuvent disparaître à tout moment.

1.2 Problématique

Cette organisation entraîne une perte de temps, des erreurs de stock, un risque de perte de dossiers et une absence de visibilité pour la direction.

1.3 Objet du document

Ce document définit les besoins, le périmètre, l'organisation, le planning et les critères de réception. Il sert de contrat entre l'équipe et le client (la direction de LogiStock SA, représentée par l'enseignant).

1.4 Objectifs du projet

Objectif général. Concevoir en équipe une application web de type ERP / CRM léger qui centralise la gestion des utilisateurs, des stocks, des ventes, des clients et des indicateurs de pilotage.

Objectifs spécifiques :

- centraliser et sécuriser les données de l'entreprise ;
- fiabiliser la gestion des stocks (stock jamais négatif, alertes de seuil) ;
- automatiser la facturation et gagner du temps ;
- fournir à la direction un tableau de bord de pilotage.

Indicateur de succès. Taux d'erreur inférieur à 1 %.

Gain attendu. Renforcer la confiance de la société et valoriser le travail réalisé par l'équipe.

2. Périmètre de la version 1

2.1 Inclus dans le périmètre

Module	Contenu
Module 1 : Authentification et RBAC	Inscription, connexion sécurisée, gestion des utilisateurs, attribution de rôles et de droits d'accès
Module 2 : Gestion des stocks	Produits, catégories, mouvements d'entrée et de sortie, historique horodaté, alertes de seuil, interdiction du stock négatif
Module 3 : Ventes et clients (CRM)	Fiches clients, commandes, validation avec décrémentation du stock, facture PDF simplifiée
Module 4 : Tableau de bord KPI	Chiffre d'affaires, produits les plus vendus, ruptures et alertes, indicateur supplémentaire (panier moyen), graphiques selon le rôle

2.2 Hors périmètre

- paiement en ligne réel ;
- application mobile native ;
- migration de l'ancien historique de données.

3. Acteurs du système

Acteur	Rôle dans l'application	Droits principaux
Visiteur	Personne non authentifiée	Inscription et connexion uniquement
Administrateur	Pilote l'activité et gère les accès	Accès complet : gestion des comptes et des rôles, tableau de bord KPI. Il hérite des droits du Gestionnaire de stock et du Commercial (généralisation entre acteurs en UML)
Gestionnaire de stock	Gère le catalogue et les quantités	Produits, catégories, mouvements de stock, alertes
Commercial	Gère les clients et les ventes	Clients, commandes, factures PDF
Membre du groupe	Acteur technique de l'équipe projet	Stories techniques : Docker, Git, tests, documentation

4. Spécifications fonctionnelles

Chaque exigence est identifiée (EF) pour faciliter le suivi et la recette. La priorité suit la méthode MoSCoW : Must (indispensable), Should (importante), Could (souhaitable).

4.1 Module 1 : Authentification et gestion des rôles (RBAC)

Réf.	Exigence	Priorité
EF1.1	Un visiteur peut créer un compte. Le mot de passe est haché avec Bcrypt avant stockage. Un email déjà utilisé est refusé avec un message clair. Les champs obligatoires sont validés côté serveur.	Must
EF1.2	Un utilisateur se connecte avec son email et son mot de passe. Un jeton JWT à durée de validité limitée est délivré. Des identifiants invalides renvoient une erreur sans révéler le champ fautif.	Must
EF1.3	Trois rôles existent : Administrateur, Gestionnaire de stock, Commercial. Chaque fonctionnalité n'est accessible qu'aux rôles autorisés. Le contrôle est effectué côté serveur par un middleware RBAC.	Must
EF1.4	Un utilisateur non authentifié reçoit un refus HTTP 401. Un utilisateur sans les droits requis reçoit un refus HTTP 403.	Must
EF1.5	L'Administrateur gère les utilisateurs et attribue les rôles.	Must

4.2 Module 2 : Gestion des stocks

Réf.	Exigence	Priorité
EF2.1	Créer, consulter, modifier et supprimer des produits. Un produit contient : référence (unique), désignation, description, catégorie, prix d'achat, prix de vente, quantité en stock, seuil d'alerte. Seuls les rôles autorisés peuvent modifier un produit.	Must
EF2.2	Gérer les catégories (création, modification, suppression). Une catégorie contenant des produits ne peut pas être supprimée.	Must
EF2.3	Enregistrer les entrées et sorties de stock. Chaque mouvement est horodaté, associé au produit et à l'utilisateur responsable, et conservé dans un historique consultable par produit. La quantité du produit est mise à jour à chaque mouvement.	Must
EF2.4	Toute opération menant à un stock négatif est refusée. Un message explicite indique la quantité disponible.	Must
EF2.5	Les produits dont la quantité est inférieure ou égale au seuil d'alerte sont signalés. La liste est mise à jour après chaque mouvement.	Must

4.3 Module 3 : Ventes et clients (CRM)

Réf.	Exigence	Priorité
EF3.1	Créer et modifier des fiches clients : identité, coordonnées, historique des commandes.	Must
EF3.2	Créer une commande liée à un client existant et contenant au moins une ligne (produit, quantité). Le total est calculé automatiquement.	Must
EF3.3	Valider une commande : les quantités sont décrémentées automatiquement. La validation est refusée si le stock est insuffisant, avec indication du produit concerné. L'opération est transactionnelle : tout est validé ou rien.	Must
EF3.4	Générer une facture simplifiée au format PDF (client, produits, quantités, total) à partir d'une commande validée. Aucune facture n'est générée pour une commande non validée.	Must

4.4 Module 4 : Tableau de bord KPI

Réf.	Exigence	Priorité
EF4.1	Chiffre d'affaires sur une période choisie (date de début et de fin). Seules les commandes validées sont comptabilisées.	Must
EF4.2	Palmarès des produits les plus vendus, trié par quantité vendue, accompagné d'un graphique.	Must
EF4.3	Liste des produits en rupture (quantité nulle) distinguée de celle des produits sous le seuil d'alerte.	Must
EF4.4	Indicateur supplémentaire : le panier moyen (chiffre d'affaires divisé par le nombre de commandes validées), documenté et justifié dans le dossier.	Should
EF4.5	Chaque utilisateur ne voit que les indicateurs autorisés pour son rôle. Le filtrage est appliqué côté serveur, pas seulement dans l'interface.	Must

4.5 Règles de gestion transversales

Réf.	Règle
RG1	Le stock d'un produit ne peut jamais être négatif.
RG2	Une commande ne peut pas être validée si le stock disponible est insuffisant.
RG3	La validation d'une commande décrémente automatiquement le stock et s'exécute dans une transaction unique (commande, lignes, stock).

Réf.	Règle
RG4	Une facture n'existe que pour une commande validée ; une commande a au plus une facture.
RG5	La référence d'un produit et l'email d'un utilisateur sont uniques.
RG6	Les mots de passe ne sont jamais stockés en clair.
RG7	Toute fonctionnalité est protégée par authentification et par contrôle de rôle côté serveur.

4.6 Cycle de vie d'une commande

États proposés : En attente, Validée, En cours de traitement, Terminée, Annulée.

- Une commande est créée à l'état « En attente ».
- Elle passe à « Validée » si le stock est suffisant.
- Elle peut être annulée depuis « En attente » ou « Validée » (annulation autorisée).
- Une commande validée peut passer à « En cours de traitement », puis à « Terminée ».

Si une commande validée est annulée, l'équipe doit définir comment rétablir les quantités en stock (remise en stock par un mouvement d'entrée ou d'ajustement tracé). Cette règle sera arrêtée en phase de conception.

5. Exigences techniques et non fonctionnelles

5.1 Architecture

- Architecture en couches, avec séparation stricte entre le frontend et le backend.
- Communication par une API REST documentée (routes, paramètres, codes de retour).
- Backend structuré selon les principes de la Clean Architecture.
- Base de données relationnelle normalisée, cohérente avec les user stories.

5.2 Sécurité

- Authentification par jeton JWT à durée de validité limitée.
- Contrôle d'accès par rôles (RBAC) appliqué au niveau de l'API.
- Hachage des mots de passe avec Bcrypt.
- Validation des données côté serveur.
- Codes de retour normalisés : 401 (non authentifié), 403 (droits insuffisants), 409 (conflit, ex. stock insuffisant).

5.3 Outils et environnement

Domaine	Exigence
Gestion de version	Git selon le modèle GitFlow : branches main, develop et feature/*. Aucun commit direct sur main.

Domaine	Exigence
Conteneurisation	Docker et Docker Compose : une seule commande démarre frontend, backend et base de données. Variables d'environnement documentées.
Génération PDF	Bibliothèque adaptée à la stack retenue (ex. pdf-lib ou pdfmake en JavaScript, iText ou OpenPDF en Java).
Tests	Tests unitaires (Jest ou PyTest selon la stack) sur le stock négatif, la validation de commande et le RBAC. Exécutés avant chaque fusion dans develop.
Documentation	README d'installation et de lancement, documentation des endpoints de l'API, scénario de démonstration.

5.4 Technologies retenues

À compléter par l'équipe avant validation (point resté ouvert dans la charte de projet) :

Composant	Choix de l'équipe
Langage et framework backend	_____
Framework frontend	_____
Système de gestion de base de données	_____
Bibliothèque PDF	_____
Outil de tests	_____

7. Backlog produit

Le backlog comprend 22 user stories pour 82 points d'effort (échelle de Fibonacci). 20 stories sont de priorité Must et 2 de priorité Should.

ID	User story	Priorité	Points	Sprint
US1.1	Inscription	Must	3	S5
US1.2	Connexion	Must	3	S5
US1.3	Contrôle des accès par rôle	Must	5	S5
US2.1	Gérer les produits	Must	5	S6
US2.2	Gérer les catégories	Must	2	S6
US2.3	Enregistrer les mouvements de stock	Must	5	S7

ID	User story	Priorité	Points	Sprint
US2.4	Empêcher le stock négatif	Must	3	S7
US2.5	Alertes de seuil	Must	3	S7
US3.1	Gérer les fiches clients	Must	3	S9
US3.2	Créer une commande	Must	5	S9
US3.3	Valider une commande	Must	8	S10
US3.4	Générer une facture PDF	Must	5	S9 à S10
US4.1	Chiffre d'affaires	Must	5	S11
US4.2	Produits les plus vendus	Must	3	S11
US4.3	Ruptures et alertes	Must	2	S11
US4.4	Panier moyen	Should	2	S11
US4.5	Affichage selon le rôle	Must	3	S11
US5.1	Conteneurisation	Must	5	S3 et S11
US5.2	Workflow Git partagé	Must	2	S3
US5.3	Revue de code croisée	Should	2	Continu
US5.4	Tests des règles critiques	Must	5	Continu
US5.5	Documentation et démonstration	Must	3	S12

Le détail des critères d'acceptation de chaque story figure dans le document « User Stories LogiStock, Groupe 6 ».

8. Organisation de l'équipe

La démarche est Agile, inspirée de Scrum. Le Product Owner et le Scrum Master participent aussi au développement.

Rôle	Membre(s)	Responsabilité
Chef de projet / Scrum Master	Checkne KAMISSOKO	Backlog, sprints et planification
Équipe Backend	Checkne KAMISSOKO, Diakaridia SIDIBE	Clean Architecture, API, règles métier, RBAC, génération PDF
Équipe Frontend	Mohamed DOUCOURE, Harouna MAIGA	Interface utilisateur et communication avec l'API
Équipe Base de	Moussa KONE, Aminata TANGARA,	Modélisation et gestion des données,

Rôle	Membre(s)	Responsabilité
données	Djenebou TRAORE	requêtes KPI

Chaque story a un responsable identifié et un relecteur différent.

Rituels : sprint de 1 à 2 séances avec objectif clair ; mêlée de séance de 15 minutes en début de séance ; rétrospective en fin de jalon.

9. Planning et jalons

Semaine	Phase	Objectif	Jalon
S1 à S4	Conception	Cahier des charges, backlog, MCD/MLD, architecture, initialisation Git et Docker	Jalon 1 (S4)
S5	Développement	Module 1 : authentification et RBAC	
S6	Développement	Module 2 : produits et catégories	
S7	Développement	Module 2 : mouvements, alertes, stock négatif	
S8	Jalon	Version alpha et revue de code	Jalon 2 (S8)
S9	Développement	Module 3 : clients, commandes, facture de base	
S10	Développement	Module 3 : validation et décrémentation, finalisation de la facture	
S11	Développement	Module 4 : KPI, graphiques, dockerisation	
S12	Jalon	Livraison finale, tests, documentation, soutenance	Jalon 3 (S12)

10. Qualité et définition de « terminé »

Une story est terminée si elle respecte ses critères d'acceptation et les quatre critères suivants :

1. le code est enregistré dans Git sur la branche develop ;
2. la fonctionnalité a été testée manuellement sur un scénario représentatif ;
3. les règles de sécurité et les autorisations RBAC sont respectées ;
4. un test unitaire couvre les règles critiques concernées.

Chaque pull request est relue par au moins un autre membre, avec vérification des points de sécurité (RBAC, validation).

11. Livrables

- application web complète (4 modules), conteneurisée avec Docker Compose ;
- code source versionné sous Git (GitFlow) ;
- dossier de conception (backlog, diagrammes UML, MCD/MLD, architecture, ADR) ;
- README d'installation et de lancement ;

- documentation de l'API (routes, paramètres, codes de retour) ;
- tests automatisés sur les règles critiques ;
- rapport de revue de code (jalon 2) ;
- jeu de données de démonstration et scénario de démonstration ;
- support de soutenance (10 min de présentation, 10 min de démonstration, 10 min de questions).

12. Critères de réception

Le projet est accepté si :

- les quatre modules sont disponibles selon le périmètre défini ;
- les rôles limitent l'accès aux fonctions et aux données autorisées ;
- les stocks ne deviennent jamais négatifs et les commandes sont bloquées si le stock est insuffisant ;
- la facture PDF est générée à partir des commandes validées ;
- les KPI sont cohérents avec les données et accessibles selon le rôle ;
- le taux d'erreur constaté est inférieur à 1 % ;
- le projet est versionné, documenté et accompagné de tests sur les règles critiques.

13. Risques et mesures

Risque	Mesure
Retard sur un jalon	Suivi à chaque séance, priorisation MoSCoW (les stories Should et Could sont repoussées en premier)
Conflits de code entre membres	GitFlow, branches feature, revue de code croisée
Incohérence des données de stock	Transactions, règles RG1 à RG3, tests unitaires
Faille de sécurité	RBAC côté serveur, Bcrypt, JWT, validation serveur, revue de sécurité
Incident lors de la démonstration	Jeu de données testé à l'avance, captures d'écran ou vidéo de secours
Dépendance à un seul membre	Chaque membre doit savoir présenter toutes les parties du projet

FIN DU DOSSIER — GROUPE 6 — LOGISTOCK SA